WEEK 11a · PHASE 5

Troubleshooting & Optimization

Spark UI · Skew · OOM · Spill · Liquid Clustering

- Spark UI: stages, tasks, timeline, metrics
- Data skew: identification and remediation
- Disk spill: causes and fixes
- OOM diagnosis and memory configuration
- Cluster startup issues
- Liquid Clustering & Predictive Optimization
- Practice Questions (10 Qs)

Table of Contents

1. Spark UI Overview 3
2. Stage Metrics & Task Details 5
3. Data Skew — Identification 7
4. Data Skew — Remediation 9
5. Shuffle Optimization 11
6. Disk Spill 12
7. OOM Diagnosis & Memory Config 13
8. Cluster Startup Issues 15
9. Liquid Clustering & Predictive Optimization 16
10. Practice Questions 17
11. Answer Key 19

WK 11a **Spark UI Overview** Navigate and interpret query performance

1. Spark UI Overview

The Spark UI is the primary tool for diagnosing performance issues in Databricks. It exposes execution plans, stage metrics, task-level statistics, and memory usage. Understanding how to navigate it is a core exam skill for Domain 6.

Spark UI Tab	Key Information	What to Look For
Jobs	All Spark jobs triggered by the application	Failed jobs, job duration, stages count
Stages	Individual stages within each job	Stage duration, shuffle read/write, input/output bytes
Tasks (within stage)	Per-task metrics	Skewed tasks (one task >> others), spill, GC time
SQL / DataFrame	Query plan: logical → physical	Exchange (shuffle), Sort, missing filters
Storage	Cached RDD/DataFrame partitions	Cache hit rate, memory vs disk cache
Executors	Per-executor memory and task metrics	GC time > 10%, OOM, dead executors
Environment	Spark config in effect	Confirm shuffle.partitions, broadcast threshold

1.1 Accessing Spark UI in Databricks

■ **Accessing Spark UI**

```
# Method 1: From a running cluster
# Compute → Clusters → [cluster name] → Spark UI tab

# Method 2: From a completed job run
# Workflows → [job run] → Tasks → [task] → Spark UI link

# Method 3: From a notebook cell
# Cell output → View (Spark jobs) → opens Spark UI in new tab

# Databricks also provides Query History in SQL Editor
# SQL Editor → Query History → [query] → Query Profile
```

1.2 Reading the DAG Visualization

The Stage DAG shows how Spark breaks a query into stages separated by shuffle boundaries. Each arrow crossing a stage boundary represents a shuffle (expensive network transfer). Minimize stages by reducing shuffles.

■ **Stage DAG symbols**

```
# Key symbols in Stage DAG:
# WholeStageCodegen — optimized fused operators (fast)
# Exchange          — shuffle boundary (expensive)
# Sort              — sort before merge join (expensive)
# BroadcastExchange — broadcast small table (fast)
# Filter            — predicate pushdown (good)
# Scan              — file scan with partition pruning (check bytes read)
```

2. Stage Metrics & Task Details

Stage metrics aggregate task-level statistics. Task details expose per-task execution, revealing skew, spill, and GC pressure.

2.1 Key Stage-Level Metrics

Metric	Location	Healthy Value	Problem Indicator
Duration	Stage list	Proportional to data size	Stage >> others → bottleneck
Input Size	Stage summary	Total bytes scanned	Very high → missing filter pushdown
Shuffle Write	Stage summary	Minimized	High → expensive shuffle, consider broadcast
Shuffle Read	Stage summary	Matches upstream write	Mismatch → data loss or error
GC Time	Executor tab	< 5% of task time	> 10% → memory pressure, OOM risk
Peak Execution Memory	Task metrics	< 80% of executor memory	Near/over limit → spill or OOM

2.2 Task-Level Skew Indicators

Skew is visible in the Task Duration column within a stage — most tasks finish in seconds while one or a few take minutes.

■ Identifying skew in Spark UI

```
# Skew signature in Spark UI:
# Tasks tab → Duration column:
#
# Task 0:  0.3s  ←■■ normal
# Task 1:  0.4s  ←■■ normal
# Task 2:  0.2s  ←■■ normal
# Task 3: 14m 32s ←■■ SKEWED TASK (one key has 90% of data)
#
# Also check: "Shuffle Read Size / Records" per task
# Skewed task reads >> other tasks
```

2.3 Summary Statistics Panel

The summary statistics panel in a stage shows min/median/max/75th/95th percentile for Duration, GC Time, Shuffle Read, Input Size. A large gap between median and max signals skew.

■ Summary stats — skew vs healthy

```
# Example — skewed stage:
# Duration:  min=0.2s  median=0.3s  max=847s  ← massive gap!
# Shuffle Read: min=1MB  median=1MB  max=48GB  ← one partition huge

# Healthy stage:
# Duration:  min=1.2s  median=1.4s  max=1.9s  ← tight distribution
# Shuffle Read: min=120MB  median=135MB  max=155MB
```

3. Data Skew — Identification

Data skew occurs when data is unevenly distributed across partitions. One partition (and its task) receives far more data than others, causing that single task to become the bottleneck.

3.1 Root Causes of Skew

Cause	Description	Example
High-cardinality hot key	One key value has far more rows than others	"NULL" customer_id on 80% of rows
Skewed join key	Join key distribution is uneven	One product_id = 90% of order rows
Date/timestamp skew	Recent data much larger than historical	Today's partition = 10× yesterday
Null key accumulation	Nulls hash to same partition in joins	NULL user_id all in one bucket
Poor partitioning column	Partition column has low cardinality	country_code = only 3 values

3.2 Diagnosing Skew Programmatically

■ **Diagnosing skew in code**

```
# Check partition sizes
from pyspark.sql.functions import spark_partition_id, count

df.groupBy(spark_partition_id()).count().orderBy("count", ascending=False).show(20)
# Output: partition 42 has 50M rows; others have ~100K

# Check key distribution
df.groupBy("customer_id").count()
  .orderBy("count", ascending=False).show(10)
# If top key has >> average count → skew

# Check null counts on join key
df.filter(col("customer_id").isNull()).count()
```

4. Data Skew — Remediation

4.1 AQE Skew Join Handling (Automatic)

Adaptive Query Execution (AQE) detects and handles skewed join partitions automatically by splitting large partitions into smaller sub-partitions. This is the first line of defense — enabled by default in Databricks.

■ AQE skew join config

```
# AQE skew join — enabled by default in Databricks
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")

# Tune thresholds if needed
spark.conf.set(
  "spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes",
  "256MB"   # partition > 256MB AND > 5x median size → skewed
)
spark.conf.set(
  "spark.sql.adaptive.skewJoin.skewedPartitionFactor", "5"
)
```

4.2 Salting — Manual Skew Fix

Salting adds a random suffix to hot keys, distributing them across multiple partitions. The small table must be exploded to match all salt values.

■ Manual salting for skew

```
from pyspark.sql.functions import rand, floor, concat, lit, explode, array

N = 10  # number of salt buckets

# Salt the large skewed DataFrame
df_large = df_large.withColumn(
  "salted_key",
  concat(col("customer_id"), lit("_"), (floor(rand()*N)).cast("int"))
)

# Explode the small lookup DataFrame to match all N salts
df_small = df_small.withColumn(
  "salt", explode(array([lit(i) for i in range(N)]))
).withColumn(
  "salted_key",
  concat(col("customer_id"), lit("_"), col("salt"))
)

# Join on salted key
result = df_large.join(df_small, on="salted_key", how="inner")
                 .drop("salted_key", "salt")
```

4.3 Filter NULL Keys Before Join

■ Filter nulls before join

```
# NULL keys all hash to the same partition → skew
# Solution: filter nulls before join, union back after

df_non_null = df.filter(col("customer_id").isNotNull())
df_nulls    = df.filter(col("customer_id").isNull())

joined = df_non_null.join(df_lookup, on="customer_id", how="left")
result = joined.unionByName(df_nulls, allowMissingColumns=True)
```

■ **Exam Focus** Exam: AQE handles most skew automatically. Manual salting is the answer when AQE is insufficient (extreme skew) or when the question asks for an explicit fix.

5. Shuffle Optimization

Shuffles are the most expensive Spark operation — they involve serializing data, writing to disk, and transferring across the network. Minimizing shuffles is key to performance.

Shuffle Optimization	How	When to Apply
Broadcast join	broadcast() hint or auto via AQE	Small table < autoBroadcastJoinThreshold
Reduce shuffle partitions	Lower spark.sql.shuffle.partitions	Small-medium datasets (AQE auto-coalesces)
Repartition before join	df.repartition(N,"key")	Pre-partition on join key to avoid re-shuffle
Avoid unnecessary groupBy	Push filters before aggregation	Filter early to reduce shuffle data volume
Coalesce vs Repartition	coalesce() reduces without full shuffle	Narrowing partitions after filter

■ repartition vs coalesce

```
# repartition vs coalesce

# repartition(N): full shuffle — distributes evenly — use to INCREASE or balance
df.repartition(200, col("customer_id"))

# coalesce(N): no shuffle — combines partitions locally — use to DECREASE
# WARNING: may create skew if partitions are unequal size
df.coalesce(10)

# Rule: repartition to increase or fix skew; coalesce to decrease cheaply
```

6. Disk Spill

Disk spill occurs when a task's data exceeds available executor memory and Spark writes overflow data to disk. Spill dramatically increases task duration due to disk I/O.

Spill Type	What It Means	Location in Spark UI
Spill (memory)	Data size in memory before spill	Stage summary → Spill (Memory) column
Spill (disk)	Actual bytes written to disk	Stage summary → Spill (Disk) column

6.1 Causes and Fixes

Cause	Symptom	Fix
shuffle.partitions too low	Few large partitions overflow memory	Increase shuffle.partitions or enable AQE
Large broadcast table	Executor OOM during broadcast	Lower autoBroadcastJoinThreshold; let sort-merge join run
Skewed partition	One task writes massive spill	Fix skew first (AQE or salting)
Insufficient executor memory	All tasks spilling	Increase executor memory / use larger worker nodes
Collecting large data to driver	collect() / toPandas() on big DF	Never collect > driver memory; use sample or limit

7. OOM Diagnosis & Memory Configuration

Out-of-Memory (OOM) errors crash executors and fail jobs. Understanding Spark's memory model is essential for diagnosing and fixing OOM.

7.1 Spark Memory Layout

Memory Region	Purpose	Config
Execution Memory	Shuffle, sort, aggregation buffers	spark.memory.fraction (default 0.6)
Storage Memory	Cached RDDs/DataFrames	Shares pool with execution memory
User Memory	UDFs, data structures in code	1 - spark.memory.fraction
Reserved Memory	Spark internal overhead	300MB fixed
Off-heap Memory	Direct memory (optional)	spark.memory.offHeap.enabled

7.2 OOM Error Types and Fixes

OOM Type	Error Message	Root Cause	Fix
Executor OOM	java.lang.OutOfMemoryError: Java heap space	Task data > executor memory	Increase executor memory; fix skew; increase partitions
Driver OOM	Driver OOM on collect()	collect()/toPandas() brings all data to driver	Never collect large DFs; use write() instead
Broadcast OOM	Could not execute broadcast	Broadcast table larger than executor memory	Disable broadcast for this table
GC overhead	GC overhead limit exceeded	Excessive GC pauses	Tune executor memory; reduce partition size

■ OOM fixes

```
# Increase executor memory (cluster config)
spark.executor.memory = 16g     # heap memory
spark.executor.memoryOverhead = 4g  # off-heap overhead

# Fix: too few partitions → large task → OOM
spark.conf.set("spark.sql.shuffle.partitions", "400")

# Fix: disable auto-broadcast for a specific table
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "-1")

# Fix: never collect large DataFrames to driver
# BAD:  df.collect()  or  df.toPandas()
# GOOD: df.write.format("delta").save("output/path")
```

7.3 GC Time as OOM Precursor

High GC time (> 10% of task duration) in the Executors tab indicates memory pressure before OOM occurs. Address it proactively.

■ GC time monitoring

```
# Check GC time: Spark UI → Executors → "GC Time" column
# > 10% of task time = memory pressure warning
# > 30% of task time = imminent OOM risk

# Options:
# 1. Increase executor memory
# 2. Reduce partition size (increase shuffle.partitions)
# 3. Unpersist cached DataFrames no longer needed:
```

```
df_cached.unpersist()
```

8. Cluster Startup Issues

Issue	Symptom	Root Cause	Fix
Cluster stuck in PENDING	Cluster never starts	Insufficient cloud VM quota or spot instance unavailability	Request quota increase; switch to on-demand; change node type
Init script failure	Cluster starts then dies	Error in cluster init script	Check init script logs in cluster event log; fix script
Library install failure	Cluster starts but library missing	Incompatible library version or PyPI unavailable	Pin library version; use cluster policy or DBR with pre-installed libs
Docker image pull failure	Container clusters fail	Private container registry auth expired	Refresh container registry credentials in cluster config
Driver OOM at startup	Cluster starts then crashes immediately	Driver memory too small for notebook state	Increase driver memory type in cluster config

■ Cluster troubleshooting commands

```
# View cluster event log:
# Compute → [cluster] → Event Log tab
# Look for: Error, Terminating, DriverDead events

# View init script logs:
# Compute → [cluster] → Configuration → Init Scripts
# Then: dbfs:/cluster-logs/<cluster_id>/init_scripts/

# Check driver logs:
# Compute → [cluster] → Driver Logs → Log4j output
```

9. Liquid Clustering & Predictive Optimization

9.1 Liquid Clustering — Recap

Feature	OPTIMIZE + ZORDER	Liquid Clustering
Activation	Explicit OPTIMIZE ZORDER BY	CLUSTER BY at table creation
Re-clustering	Full file rewrite	Incremental — only new/changed data
Max cluster columns	~4 practical limit	No hard limit
Partition compatibility	Works with partitioned tables	Replaces partitioning; do not combine
DBR requirement	All versions	DBR 13.3 LTS+
Best for	Existing tables; infrequent updates	New tables; frequent writes

■ Liquid Clustering setup & OPTIMIZE

```
-- Create table with Liquid Clustering
CREATE TABLE catalog.silver.orders
CLUSTER BY (customer_id, order_date)
AS SELECT * FROM catalog.bronze.orders;

-- Trigger incremental clustering
OPTIMIZE catalog.silver.orders;
-- No ZORDER needed — clustering is defined at table level

-- Inspect clustering columns
DESCRIBE DETAIL catalog.silver.orders;
-- clusteringColumns: ["customer_id","order_date"]
```

9.2 Predictive Optimization

Predictive Optimization is a Databricks feature that automatically runs OPTIMIZE and VACUUM on Unity Catalog tables based on usage patterns and data statistics — no manual scheduling needed.

Predictive Optimization	Detail
What it does	Auto-runs OPTIMIZE (compaction) and VACUUM based on table statistics and query patterns
Enabled on	Unity Catalog managed tables (DBR 12.2 LTS+)
Configuration	Enable at account/metastore level or per catalog/schema/table
Benefit	Eliminates need for manual OPTIMIZE jobs; reduces storage costs via VACUUM
Exam tip	Know it exists and what it automates — not a manual-run command

■ Predictive Optimization enablement

```
-- Enable Predictive Optimization on a schema
ALTER SCHEMA catalog.silver
SET TBLPROPERTIES (delta.enablePredictiveOptimization = true);

-- Enable on a specific table
ALTER TABLE catalog.silver.orders
SET TBLPROPERTIES (delta.enablePredictiveOptimization = true);

-- Check optimization history
DESCRIBE HISTORY catalog.silver.orders;
```

QUIZ **Practice Questions** Week 11a — Troubleshooting & Optimization

10. Practice Questions

1. In the Spark UI Stages tab, most tasks in a stage complete in 2 seconds, but one task takes 18 minutes. What is the most likely cause?

A) The task ran on a slow executor node

B) Data skew — one partition contains disproportionately more data than others

C) shuffle.partitions is set too high

D) The query is missing an index

2. Which Spark UI tab shows "Spill (Memory)" and "Spill (Disk)" columns for each stage?

A) Jobs tab

B) SQL tab

C) Stages tab

D) Executors tab

3. An executor's GC Time is 35% of its total task time. What does this indicate and what should be done?

A) Normal behavior — GC always uses ~30% of time

B) Memory pressure warning — increase executor memory or reduce partition size

C) The executor is performing disk I/O — add more disks

D) The cluster is undersized for the number of users

4. A job fails with "java.lang.OutOfMemoryError: Java heap space" on the driver. What is the most likely cause?

A) An executor ran out of memory during a shuffle

B) df.collect() or df.toPandas() was called on a very large DataFrame

C) The broadcast join threshold is too high

D) shuffle.partitions is set too low

5. What is the difference between Spill (Memory) and Spill (Disk) in Spark UI?

A) Spill (Memory) is bytes read from disk; Spill (Disk) is bytes written

B) Spill (Memory) is the data size in memory before spill; Spill (Disk) is actual bytes written to disk

C) Both measure the same thing — disk and memory are interchangeable terms here

D) Spill (Memory) means off-heap spill; Spill (Disk) means on-heap spill

6. A Delta table is frequently filtered on customer_id and region. The table uses OPTIMIZE ZORDER BY (customer_id). What additional action improves performance for region filters?

A) Add a second OPTIMIZE ZORDER BY (region) run — it replaces the first

B) Add region to the ZORDER: OPTIMIZE ZORDER BY (customer_id, region)

C) Create a separate table partitioned by region

D) ZORDER only supports one column — use Liquid Clustering instead

7. AQE (Adaptive Query Execution) automatically handles which of the following? (Choose the BEST answer)

A) Only auto-broadcasts small tables

B) Auto-coalesces small shuffle partitions, converts sort-merge to broadcast join, and handles skewed joins

C) Only coalesces partitions — join optimization requires explicit config

D) Auto-adds ZORDER to Delta tables based on query patterns

8. A cluster is stuck in PENDING state for 20 minutes. What is the most likely cause?

A) The init script has a Python syntax error

B) Insufficient VM quota or spot instance unavailability in the cloud provider

C) The Databricks workspace is under maintenance

D) The cluster policy restricts the selected node type

9. What does Predictive Optimization automatically run on Unity Catalog tables?

A) ANALYZE and EXPLAIN for query planning

B) OPTIMIZE (compaction) and VACUUM (file cleanup) based on usage patterns

C) ZORDER BY on commonly filtered columns

D) MERGE INTO for incremental updates

10. A Liquid Clustering table is defined with CLUSTER BY (order_date). After many inserts, clustering quality degrades. What command re-clusters it?

A) OPTIMIZE table ZORDER BY (order_date)

B) CLUSTER TABLE table BY (order_date)

C) OPTIMIZE table — Liquid Clustering is triggered automatically

D) RECLUSTER table ON (order_date)

ANSWERS **Answer Key** Week 11a Practice Questions

11. Answer Key

#	Ans	Explanation
1	(B)	A single task taking much longer than all others is the classic symptom of data skew — one partition key has far more rows, causing that task to process most of the data.
2	(C)	The Stages tab shows per-stage aggregate metrics including Spill (Memory) — data size before spill — and Spill (Disk) — bytes written to disk due to memory overflow.
3	(B)	GC Time > 10% indicates memory pressure; 35% is a severe warning of imminent OOM. Fix: increase executor memory, reduce partition size by raising shuffle.partitions, or unpersist unused cached DataFrames.
4	(B)	Driver OOM typically occurs when collect() or toPandas() is called — this pulls ALL data to the driver node. Fix: write the DataFrame to storage instead of collecting it.
5	(B)	Spill (Memory) = the serialized size of data that needed to be spilled (in memory representation). Spill (Disk) = actual bytes written to disk after compression. Disk < Memory due to compression.
6	(B)	ZORDER BY supports multiple columns: OPTIMIZE ZORDER BY (customer_id, region). Both columns will be used for data co-location and skipping. Practical limit is ~4 columns.
7	(B)	AQE handles three main optimizations: (1) coalescing small shuffle partitions, (2) converting sort-merge joins to broadcast joins when one side is smaller than threshold at runtime, (3) skewed join handling.
8	(B)	PENDING means Databricks is waiting for cloud VMs to be provisioned. Common causes: VM quota exceeded, spot instance unavailability, or the selected node type is temporarily unavailable in the region.
9	(B)	Predictive Optimization automatically runs OPTIMIZE (to compact small files) and VACUUM (to remove old files) on Unity Catalog managed tables, eliminating the need for manual scheduled maintenance jobs.
10	(C)	For Liquid Clustering tables, running OPTIMIZE (without ZORDER BY) triggers incremental re-clustering based on the CLUSTER BY columns defined at table creation. No column specification needed.

12. Week 11a Cheat Sheet

Topic	Key Fact
Spark UI skew indicator	One task >> all others in Duration column within a stage
Summary stats skew	Large gap between median and max Duration/Shuffle Read
AQE skew join	Auto-splits large partitions; enabled by default (skewJoin.enabled=true)
Salting	Add random suffix to hot key; explode small table to match N salts
repartition vs coalesce	repartition=full shuffle, balanced; coalesce=no shuffle, may skew
Spill (Memory)	Data size in memory before overflow; Spill (Disk) = bytes written to disk
Driver OOM cause	collect() / toPandas() on large DataFrame
Executor OOM cause	Task data > executor heap; fix: more memory or more partitions
GC Time > 10%	Memory pressure warning — increase memory or reduce partition size
Cluster PENDING	Cloud VM quota exceeded or spot unavailable
Liquid Clustering re-cluster	OPTIMIZE — no ZORDER needed
Predictive Optimization	Auto-runs OPTIMIZE + VACUUM on Unity Catalog tables
DESCRIBE DETAIL	Shows numFiles, averageFileSize, clusteringColumns

BONUS **Advanced Optimization Patterns** Lab & Exam Deep Dive

A. Query Plan Reading

Reading the physical query plan helps identify bottlenecks before execution.

■ EXPLAIN FORMATTED query plan

```
-- View the physical execution plan
EXPLAIN FORMATTED SELECT * FROM silver.orders o
JOIN silver.customers c ON o.customer_id = c.customer_id
WHERE o.order_date >= "2024-01-01";

-- Key plan nodes to look for:
-- BroadcastHashJoin     → small table broadcast (fast)
-- SortMergeJoin         → both tables shuffled (expensive)
-- Exchange hashpartitioning → shuffle boundary
-- FileScan              → check numFiles and pushed filters
-- Filter                → predicate pushed down (good)

-- PySpark equivalent
df.explain("formatted")  # or df.explain(True)
```

Plan Node	Meaning	Action
BroadcastHashJoin	Small table broadcast — fast join	Good — no action needed
SortMergeJoin	Both tables shuffled and sorted	Consider broadcast hint for small side
Exchange hashpartitioning	Shuffle — expensive network transfer	Minimize; use broadcast or pre-partition
PartialAggregate	Map-side pre-aggregation	Good — reduces shuffle data
FileScan with pushed filters	Partition pruning active	Good — verify pushed filters exist
FileScan without pushed filters	Reading all files	Check WHERE clause; ZORDER/Cluster on filter cols

B. Performance Tuning Decision Tree

Symptom	First Check	Primary Fix
One task >> all others in duration	Spark UI Stages → Task Duration column	AQE skew join or manual salting
Stage has high Spill (Disk)	Spark UI Stages → Spill columns	Increase shuffle.partitions or executor memory
GC Time > 10% in Executors tab	Executors → GC Time column	Increase executor memory or reduce partition size
Driver OOM on collect()	Code review	Replace collect()/toPandas() with write()
Query reads all files despite filter	EXPLAIN FORMATTED → FileScan node	Add ZORDER BY or Liquid Clustering on filter columns
Cluster stuck in PENDING	Cluster event log	Check VM quota; switch node type; on-demand vs spot
200 tiny shuffle tasks on small data	shuffle.partitions default=200	AQE auto-coalesces; or set shuffle.partitions lower
Slow broadcast join	Memory usage on executors	Lower autoBroadcastJoinThreshold or disable broadcast
Many small files (numFiles > 1000)	DESCRIBE DETAIL table	Run OPTIMIZE; enable Predictive Optimization

C. Domain 6 Exam Keywords Quick Map

Keyword	Points To
"one task takes much longer than others"	Data skew — AQE skew join or salting
"Spill (Disk) in Stages tab"	Disk spill — increase shuffle.partitions or executor memory
"GC Time > 10%"	Memory pressure — increase executor memory or unpersist cache
"java.lang.OutOfMemoryError on driver"	collect() / toPandas() on large DF
"java.lang.OutOfMemoryError on executor"	Task data > heap — more memory or more partitions
"BroadcastHashJoin in query plan"	Small table broadcast — fast; auto or via hint
"SortMergeJoin in query plan"	Expensive shuffle — consider broadcast if one side small
"OPTIMIZE ZORDER BY"	Co-locate data for filter cols; data skipping
"CLUSTER BY at creation"	Liquid Clustering — incremental re-clustering
"OPTIMIZE without ZORDER"	Liquid Clustering re-cluster or plain compaction
"Predictive Optimization"	Auto OPTIMIZE + VACUUM on UC managed tables
"Cluster PENDING for 20+ min"	VM quota exceeded or spot instance unavailable
"repartition vs coalesce"	repartition=full shuffle/balanced; coalesce=no shuffle/may skew

- AQE is enabled by default in Databricks — handles most skew and coalescing automatically
- Manual salting is needed for extreme skew AQE cannot handle
- Spill (Memory) >= Spill (Disk) because disk data is compressed
- NEVER collect() large DataFrames to driver — always write to storage
- Liquid Clustering: CLUSTER BY at creation; OPTIMIZE triggers incremental re-cluster
- Predictive Optimization: auto-runs OPTIMIZE + VACUUM — no manual schedule needed